

## 目標

相補フィルタを実装し、ESKF の推定値と比較する

- ジャイロ積分のドリフト問題を理解
- 相補フィルタで加速度センサとジャイロを融合
- Teleplot で CF vs ESKF をリアルタイム比較

# 姿勢の表現 — ZYX オイラー角

## オイラー角とは

3次元の姿勢（回転）を3つの角度で表現する方法の一つ

| 記号       | 名称         | 回転軸     | 意味    |
|----------|------------|---------|-------|
| $\phi$   | Roll（ロール）  | $X''$ 軸 | 左右の傾き |
| $\theta$ | Pitch（ピッチ） | $Y'$ 軸  | 前後の傾き |
| $\psi$   | Yaw（ヨー）    | $Z$ 軸   | 機首方向  |

## StampFly の規約: ZYX オイラー角

回転順序:  $Z$  軸 ( $\psi$ )  $\rightarrow$   $Y'$  軸 ( $\theta$ )  $\rightarrow$   $X''$  軸 ( $\phi$ )

航空宇宙分野で標準的な **Tait–Bryan 角**。ジンバルロック ( $\theta = \pm 90^\circ$ ) に注意。

# オイラー角微分とジャイロ角速度の関係

## ジャイロの測定値 $\omega$ からオイラー角の微分を求める

ジャイロはボディ座標系の角速度  $\omega = [\omega_x, \omega_y, \omega_z]^T$  を測定する。オイラー角の微分とは**一致しない**。現在の姿勢に依存する変換が必要:

$$\dot{\phi} = \omega_x + \omega_y \sin \phi \tan \theta + \omega_z \cos \phi \tan \theta$$

$$\dot{\theta} = \omega_y \cos \phi - \omega_z \sin \phi$$

$$\dot{\psi} = \omega_y (\sin \phi / \cos \theta) + \omega_z (\cos \phi / \cos \theta)$$

## 離散積分（毎ステップ更新）

$$\phi_{k+1} = \phi_k + \dot{\phi}_k \Delta t, \quad \theta_{k+1} = \theta_k + \dot{\theta}_k \Delta t, \quad \psi_{k+1} = \psi_k + \dot{\psi}_k \Delta t$$

**注意:** 現在の  $(\phi, \theta)$  に依存  $\rightarrow$  毎ステップ再計算。  $\theta = \pm 90^\circ$  で  $\cos \theta = 0$  (ジンバルロック)

# 加速度からのオイラー角算出 (1) — 座標変換

静止時、加速度センサは重力の**反力**を測定する

NED 座標系の重力  $[0, 0, g]^T$  に対し、センサ出力は  $\mathbf{R}^T [0, 0, -g]^T$ :

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = \mathbf{R}^T \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = \begin{bmatrix} g \sin \theta \\ -g \sin \phi \cos \theta \\ -g \cos \phi \cos \theta \end{bmatrix}$$

- $a_x$  は**ピッチ角  $\theta$  のみ**に依存 ( $\phi$  を含まない)
- $a_y$  と  $a_z$  は**ロール角  $\phi$  とピッチ角  $\theta$  の両方**に依存
- **ヨー角  $\psi$**  はどの成分にも現れない (重力は鉛直  $\rightarrow \psi$  の情報なし)

**前提:** 機体が静止または等速運動中であること。加速中は重力以外の成分が混入しノイズとなる。

# 加速度からのオイラー角算出 (2) — Roll と Pitch の導出

Roll  $\phi$  の算出:  $(-a_y)$  と  $(-a_z)$  の比

$(-a_y)/(-a_z)$  をとると  $g \cos \theta$  が約分され  $\theta$  が消える:

$$\frac{-a_y}{-a_z} = \frac{g \sin \phi \cos \theta}{g \cos \phi \cos \theta} = \frac{\sin \phi}{\cos \phi} = \tan \phi \quad \Rightarrow \quad \boxed{\phi = \text{atan2}(-a_y, -a_z)}$$

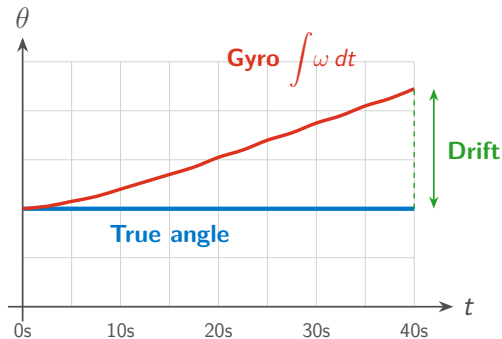
Pitch  $\theta$  の算出:  $a_x$  と  $\sqrt{a_y^2 + a_z^2}$

$a_y^2 + a_z^2 = g^2 \cos^2 \theta (\sin^2 \phi + \cos^2 \phi) = g^2 \cos^2 \theta$  より:

$$\frac{a_x}{\sqrt{a_y^2 + a_z^2}} = \frac{g \sin \theta}{g \cos \theta} = \tan \theta \quad \Rightarrow \quad \boxed{\theta = \text{atan2}(a_x, \sqrt{a_y^2 + a_z^2})}$$

**Yaw  $\psi$ :** 加速度のみでは算出不可 (磁気センサが必要)   **長所:** ドリフトなし   **短所:** 振動ノイズ大

# ジャイロドリフト問題 / Gyro Drift Problem

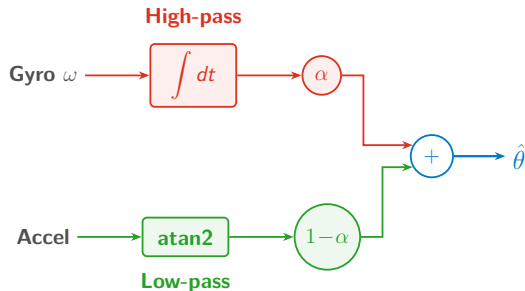


- ジャイロは**角速度**  $\omega$  を測定
- 角度  $\theta = \int \omega dt$  (積分が必要)
- バイアスが蓄積 → ドリフト増大

## 問題

ジャイロ単体では長時間の姿勢推定が不可能

# 相補フィルタ / Complementary Filter



## 数式

$$\hat{\theta}_k = \alpha (\hat{\theta}_{k-1} + \omega \Delta t) + (1 - \alpha) \theta_{\text{accel}}$$

- $\alpha = 0.98$  (ジャイロ 98% + 加速度 2%)
- ジャイロ: 短期精度◎ / 長期ドリフト☒
- 加速度: 長期安定◎ / 振動ノイズ☒

# 姿勢推定 API

| 関数                                      | 説明                   | 単位               |
|-----------------------------------------|----------------------|------------------|
| <code>gyro_x/y/z()</code>               | 角速度 (Roll/Pitch/Yaw) | rad/s            |
| <code>accel_x/y/z()</code>              | 加速度 (X/Y/Z)          | m/s <sup>2</sup> |
| <code>estimated_roll/pitch/yaw()</code> | ESKF 推定姿勢角           | rad              |
| <code>print(fmt, ...)</code>            | シリアル出力 (Teleplot 対応) | —                |

## 実習で使う組み合わせ

ジャイロ + 加速度 → 相補フィルタで姿勢を自作  
`estimated_roll()` → ESKF の推定値と比較

# 実習: 相補フィルタ + Teleplot / Hands-on

user\_code.cpp (excerpt)

```
1 #include "workshop_api.hpp"
2 #include <cmath>
3 static float cf_roll = 0.0f, cf_pitch = 0.0f;
4 void setup() { ws::print("Lesson 9: Attitude Estimation"); }
5 void loop_400Hz(float dt) {
6     float gx = ws::gyro_x(), gy = ws::gyro_y();
7     float ax = ws::accel_x(), ay = ws::accel_y(),
8         az = ws::accel_z();
9     float accel_roll = atan2f(ay, az);
10    float accel_pitch = atan2f(-ax, az);
11    constexpr float alpha = 0.98f;
12    cf_roll = alpha*(cf_roll + gx*dt) + (1-alpha)*accel_roll;
13    cf_pitch = alpha*(cf_pitch + gy*dt) + (1-alpha)*accel_pitch;
14    // Teleplot output (VSCode Teleplot extension)
15    ws::print(">cf_roll:%.1f", cf_roll*57.3f);
16    ws::print(">eskf_roll:%.1f", ws::estimated_roll()*57.3f);
17 }
```

# Teleplot によるリアルタイム可視化

## Teleplot フォーマット

>変数名: 値 をシリアルに出力するだけ

```
1 // Teleplot format: >name:value
2 ws::print(">cf_roll:%.2f", cf_roll * 57.3f);
3 ws::print(">cf_pitch:%.2f", cf_pitch * 57.3f);
4 ws::print(">eskf_roll:%.2f",
5           ws::estimated_roll() * 57.3f);
```

**注意:** 400 Hz 全 tick で出力するとシリアル帯域に負荷。4 tick 毎 (100 Hz) にデシメーション推奨

## セットアップ

- 1 VSCode 拡張:  
alexnesnes.teleplot  
をインストール
- 2 Teleplot パネルでシリアルポートを選択
- 3 自動でグラフ化される

# Madgwick フィルタ / Madgwick Filter

## 勾配降下法によるクォータニオン推定

チューニングパラメータが  $\beta$  の **1つだけ** — シンプルかつ高精度

- 状態:  $\mathbf{q} = [q_0, q_1, q_2, q_3]$

- 予測:  $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}$

- 補正勾配:

$$\nabla f = \mathbf{J}^T(\hat{\mathbf{q}}, \hat{\mathbf{d}}) \mathbf{f}(\hat{\mathbf{q}}, \hat{\mathbf{d}})$$

$\mathbf{f}$ : 加速度/磁気の目的関数、 $\mathbf{J}$ : そのヤコビアン、 $\beta$ : 補正ゲイン ( $\approx 0.04$ )

### 更新則

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \Delta t \left( \frac{1}{2} \mathbf{q}_t \otimes \boldsymbol{\omega} - \beta \frac{\nabla f}{|\nabla f|} \right)$$

## 非線形カルマンフィルタの5ステップ

状態ベクトル例:  $\mathbf{x} = [\phi, \theta, \psi, b_{gx}, b_{gy}, b_{gz}]^T$

| ステップ    | 数式                                                                                            | 意味          |
|---------|-----------------------------------------------------------------------------------------------|-------------|
| 状態予測    | $\hat{\mathbf{x}}^- = f(\hat{\mathbf{x}}, \mathbf{u})$                                        | ジャイロで姿勢を積分  |
| 共分散予測   | $\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^T + \mathbf{Q}$                                | 不確かさの伝播     |
| カルマンゲイン | $\mathbf{K} = \mathbf{P}^-\mathbf{H}^T(\mathbf{H}\mathbf{P}^-\mathbf{H}^T + \mathbf{R})^{-1}$ | 予測と観測の信頼度配分 |
| 状態更新    | $\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K}(\mathbf{z} - h(\hat{\mathbf{x}}^-))$      | 観測で補正       |
| 共分散更新   | $\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$                                | 不確かさを縮小     |

# ESKF (誤差状態カルマンフィルタ)

## StampFly で実際に使われている 15 状態 ESKF

### 名目状態 + 誤差状態の分離アーキテクチャ

誤差状態ベクトル (15 states) :

$$\delta \mathbf{x} = [\delta \mathbf{p}, \delta \mathbf{v}, \delta \boldsymbol{\theta}, \delta \mathbf{b}_g, \delta \mathbf{b}_a]^T$$

- 予測: 名目状態はジャイロ/加速度で直接積分
- 誤差共分散のみ伝播

利点: 誤差状態は常に小さい → 線形化が正確

観測更新:

- 気圧 → 高度補正
- ToF → 対地高度補正
- 磁気 → ヨー補正
- 光学フロー → 速度補正

# 推定手法の比較 / Estimator Comparison

|        | 相補フィルタ         | Madgwick       | EKF            | ESKF           |
|--------|----------------|----------------|----------------|----------------|
| 計算量    | 極小             | 小              | 中              | 中～大            |
| パラメータ数 | 1 ( $\alpha$ ) | 1 ( $\beta$ )  | <b>Q, R</b> 行列 | <b>Q, R</b> 行列 |
| 推定対象   | Roll/Pitch     | R/P/Yaw        | 姿勢+バイアス        | 位置/速度/姿勢/バイアス  |
| 使用センサ  | Gyro+Accel     | Gyro+Accel+Mag | Gyro+Accel+Mag | 全センサ (6 種)     |
| 精度     | ○              | ◎              | ◎              | ◎◎             |

**StampFly のデフォルト:** ESKF (15 状態) を使用。このレッスンでは相補フィルタを自作し、ESKF と比較して理解を深める。

# チェックポイント / Checkpoint

## 確認事項

- ☐ 手で傾けると CF のロール・ピッチが変化する
- ☐ CF と ESKF の値が概ね一致する (Teleplot で確認)
- ☐  $\alpha$  を変えて応答の違いを観察
- ☐ Madgwick / EKF / ESKF の特徴を説明できる

## 次のレッスン

Lesson 10: ws:: API リファレンス

# Lesson 10

**ws:: API リ  
ファレンス**

ws:: API Reference

StampFly Workshop